

GUI64

API Documentation

Programmer's Reference for GUI64 Applications



API version: 1.0
Public API base: \$5F00
Application space: \$B000-\$CFFF
CPU: NMOS 6502
Assembler syntax: ACME
API entries: 33

Contents

1	Introductory Notes	2
2	Creating Windows, Controls, Menus, and Dialogs	2
2.1	Windows	2
2.2	Controls	3
2.3	Menus	5
2.4	Dialogs	5
3	Character set	6
4	Two Designs	6
5	Sprites and SID	6
6	Frame and Timer API	7
6.1	Frame API	7
6.2	Timer API	8
6.3	Timer and Frame Handler Comparison	8
7	Public Jump Table	9
8	Documentation of API Calls	10
8.1	Window API	10
8.2	Control API	12
8.3	Application Extensions	14
8.4	Timer API	15
8.5	Frame API	15
8.6	System API	16
8.7	Menu and Mouse API	16
8.8	Dialog API	17
8.9	Low-level Buffer Helpers	18
9	Hello World Application	18

1 Introductory Notes

GUI64 exposes its application interface through a sequence of three-byte JMP entries beginning at **\$5F00**. These routines are given in Section 7 alongside with their particular explanation. However, a programmer should always use the most recent version of the file

gui64.inc.asm

which contains these addresses and many other constants definitions. It can always be found in the *Apps* folder in the GUI64 Github repository (<https://github.com/WebFritzi/GUI64/>).

The current GUI64 application area is:

\$B000 ... \$CFFF

Warning!

GUI64 uses zero-page memory extensively. In particular, temporary pointers such as **\$02/\$03**, **\$FB/\$FC**, and **\$FD/\$FE** must be treated as volatile across API calls unless a routine explicitly documents otherwise.

GUI64 API architecture

- 16-bit addresses are passed with **X** as low byte and **Y** as high byte (**Addr(X/Y)**).
- The current window is mirrored in the static window structure at **\$10-\$1F**.
- The current control is mirrored in the static control structure at **\$20-\$2F**.
- `wndParam0/wndParam1` carry window-procedure event information.
- Carry is used by several yes/no-routines as the Boolean result.
- API routines may alter **A**, **X**, **Y**, flags, and GUI64 scratch memory in zero page.

2 Creating Windows, Controls, Menus, and Dialogs

The GUI64 API routines for creating and configuring windows, controls, menus, and dialogs. These interface elements are usually defined by static data structures and can be adjusted at runtime after creation. This chapter describes the basic workflow for creating these elements, modifying their properties, and displaying them on screen.

2.1 Windows

A GUI64 window in C64 memory is defined by a 16 byte structure containing the following properties (the “Reserved”-entries are used by GUI64 internally and should not be modified by an app):

Offset	Name	Meaning
0	CurrentWindow	Window identifier handle/index
1	WindowType	Window type (must be at least 50)
2	Reserved	-
3	WindowBits	Window property flags
4	WindowPosX	X position
5	WindowPosY	Y position
6	WindowWidth	Width
7	WindowHeight	Height
8	WindowTitleStr	16-bit pointer to string in title bar
10	Reserved	-
12	Reserved	-
13	Reserved	-
14	WindowProc	16-bit window-procedure pointer

Window bits

The window bits are (see `gui64.inc.asm`) `BIT_WND_HASMENU` (1), `BIT_WND_RESIZABLE` (2), `BIT_WND_FIXEDWIDTH` (4), `BIT_WND_FIXEDHEIGHT` (8), `BIT_WND_CANMAXIMIZE` (16) (WIN only), `BIT_WND_CANMINIMIZE` (32) (WIN only), `BIT_WND_ISMINIMIZED` (64), `BIT_WND_ISMAXIMIZED` (128) (WIN only)

Window proc

An important part of a window is its window proc. This routine handles user input events like mouse/joystick or keyboard actions. When the program pointer enters the window proc, the event code can be read from `wndParam0` (see `gui64.inc.asm`). The events are (see `gui64.inc.asm`):

`EC_RBTNPRESS`, `EC_RBTNRELEASE`, `EC_LBTNPRESS`, `EC_LBTNRELEASE`, `EC_MOUSEMOVE`, `EC_DBLCLICK`, `EC_SCROLLWHEELDOWN`, `EC_SCROLLWHEELUP`, `EC_KEYPRESS`, and `EC_SHUTDOWN`.

The other parameter `wndParam1` contains the **program mode** that was active when the event was triggered. The program modes are (see `gui64.inc.asm`) `PM_NORMAL` (0), `PM_MENU` (1), and `PM_DIALOG` (255). For example, when the user selects a menu item of the window's menu, the window proc receives an event with `EC_LBTNPRESS` in `wndParam0` and `PM_MENU` in `wndParam1`. The program mode can be read at any time from `ProgramMode` (see `gui64.inc.asm`).

Note: In each window proc, apps should call `GUI_StdWndProc` which handles events sent to the GUI64 standard controls. For example, when the user clicks on a button, the standard window proc forwards the event to the standard button action routine.

Creation

A window is created by either using `GUI_CreateWindow` or `GUI_CreateWindowEx`. While the first just creates a window and hence only expects window data, the second creates a window with controls. Therefore, there must be controls data provided right after the window data. Window data must always be provided in the following form:

- window type identifier (value must be at least 50), bits, xPos, yPos, width, height, 16-bit pointer to title string, 16-bit pointer to window proc

Each of the first 6 entries is a byte. The last two are words (pointers in little endian format).

2.2 Controls

Controls are graphical elements in a window. There are 13 standard controls in GUI64 that an app may use. Some of them are dynamic (action) and react to user input, some are static (no action). The list of controls is:

ID	Type identifier	Control	Action
1	CT_MENUBAR	Menubar	Yes
2	CT_BUTTON	Button	Yes
3	CT_LISTBOX	Listbox	Yes
5	CT_COLORPICKER	Colorpicker	Yes
6	CT_RADIOBUTTONGROUP	Radiobutton group	Yes
7	CT_UPDOWN	Up/down control	Yes
8	CT_EDIT_SL	Single-line edit	Yes
10	CT_CHECKBOX	Checkbox	Yes
31	CT_LABEL	Label	No
32	CT_LABEL_ML	Multi-line label	No
33	CT_FRAME	Frame	No
34	CT_PROGRESSBAR	Progressbar	No
35	CT_COLBOXLABEL	Colorbox label	No

The control type identifier is just a replacement of the ID (e.g., `CT_MENUBAR` = 1).

Each control in a window has properties, provided by a 16 byte structure in memory. Based on this structure, GUI64 knows how to paint the control. The entries of this structure are:

Offset	Name	Meaning
0	ControlIndex	Control index
1	ControlType	Control type (must be at least 50)
2	ControlColor	Color
3	ControlPosX	X position
4	ControlPosY	Y position
5	ControlWidth	Width
6	ControlHeight	Height
7	ControlBits	Control property flags
8	ControlHilIndex	Highlighted item index
9	ControlTopIndex	Top item / control-specific value
10	ControlNumStr	Number of strings
11	ControlStrings	16-bit string/string-list pointer
13	-	Control-specific
14	ControlBitsEx	Extended control flags
15	-	Control-specific

For some controls, the struct members 9 - 14 are used for something else:

Offset	Name	Meaning
10	UpDown_LowerLimit	Lower limit of UpDown value
11	UpDown_UpperLimit	Upper limit of UpDown value
12	UpDown_DigitLo	Low digit of UpDown control
13	UpDown_DigitHi	Hi digit of UpDown control
10	ProgressBar_MaxLo	Low byte of maximal value of progress bar
11	ProgressBar_MaxHi	High byte of maximal value of progress bar
12	ProgressBar_ValLo	Low byte of value of progress bar
13	ProgressBar_ValHi	Low byte of value of progress bar
9	EditSL_CaretPos	Position of caret in single-line edit control
10	EditSL_MaxStrLen	Maximal string length in single-line edit control
14	EditSL_Forbidden	16-bit pointer to forbidden chars

Selection and modification

In GUI64, windows and controls are *selected*. This means that window or control structures are copied from the heap to zero page (**\$10 - \$1f** for a window and **\$20 - \$2f** for a control). Once selected, a window or control is the *current* window or control. You select windows and controls by using [GUI_SelectWindow](#) or [GUI_SelectControl](#), respectively. After selecting a window or control, any changes made to its zero-page structure must be written back to the heap using [GUI_UpdateWindow](#) or [GUI_UpdateControl](#) for the changes to take effect.

Creation

There are two options to create a control. The first is using [GUI_CreateWindowEx](#), where a window is created alongside its controls. Static control information data must always be provided in the following form:

- control type identifier (value must be at least 50), xPos, yPos, width, height, null terminated string

Each entry is a byte. The null-terminated string consists of chars (Petscii) and might be empty (i.e., 0).

When you use [GUI_CreateWindowEx](#) for creating the controls in your window, you provide the window information data at **Addr(X/Y)**, followed by the list of controls information data. Don't forget to add a terminating zero at the end of the list of controls, so GUI64 knows that this was the last control in the list.

The second option to create a control is to use [GUI_AddControl](#). This adds a control with control information of the above form at **Addr(X/Y)** to the current window.

Custom controls

Programmers may register new control types. Custom control type identifiers start at 50. The new controls are added as described above. Before doing so, however, the programmer must inform GUI64 that custom controls will be added. For this, pointers to two routines have to be provided:

- one routine for painting the new controls
- one routine for defining the default behavior of the new controls (event reaction)

The programmer registers these two routines via `GUI_SetPaintCtrlsRoutine` and `GUI_SetCtrlActionsRoutine`, respectively. When the program counter enters them, the **X** register contains the control type identifier. In the case of the PaintCtrls routine, `$fd/$fe ($02/$03)` points to the upper left corner of the control in the paint (color) buffer. A control is painted by storing character indices in `$FD/$FE (sta ($fd),y)` and color indices in `$02/$03 (sta ($02),y)`. To get to the next row, one may make use of the API calls `GUI_AddBufWidthToFD` and `GUI_AddBufWidthTo02`, respectively. In the CtrlActions routine, you may use `wndParam0` and `wndParam1` in the same way as in the window proc (see above). If you only use one instance of a custom control, you may alternatively define the control's event behavior in the parent windows's window proc.

2.3 Menus

In GUI64, any window may possess a menubar. This is indicated in the window bits (`BIT_WND_HASMENU`). In the WIN version of GUI64, the menubar is placed inside the window right below the title bar, while it is placed in the system bar at the top of the screen in the MAC version. Therefore, if a window possesses a menubar, its height in the WIN version should be one unit larger than in the MAC version to display the same content. The programmer can take care of this by using the API call `GUI_GetDesign`.

A menubar is in fact a control whose control index should always be zero. As its position is not relevant, the `xPos` and `yPos` values in the control information data are used as a pointer to a list of 16-bit pointers to drop-down menus. Each drop-down menu is defined as a structure of the following form:

- menu ID, maximal string length of menu items, number of menu items, list of null-terminated strings

Menu IDs start at **10**.

Here is an example from the demo app *Sweepy*:

```
; Constants
ID_MENU_GAME = 10
ID_MENU_HELP = 11
...
...
...
; Control definition
!byte CT_MENUBAR, <MSMenubar, >MSMenubar, 0, 0
!pet 0
...
...
...
; Definition of menu bar
MSMenubar      !word Menu_MS_Game, Menu_MS_Help
Str_MSMenubar  !pet "Game",0,"?",0
; Definition of menus
; Format: ID, max_str_len, item_count, StringList
Menu_MS_Game    !pet ID_MENU_GAME,4,2,"New",0,"Quit",0
Menu_MS_Help    !pet ID_MENU_HELP,5,1,"About",0
```

2.4 Dialogs

A dialog in GUI64 is a window which sits on top of all other windows and which cannot be minimized, maximized, nor resized. Moreover, a dialog window has the complete focus. That is, if a dialog window is shown, the user can only make clicks inside this very window. All other actions are ignored. When a dialog is shown, the program mode (`ProgramMode`) is set to `PM_DIALOG (= $ff)`.

There are three system dialogs an app can use:

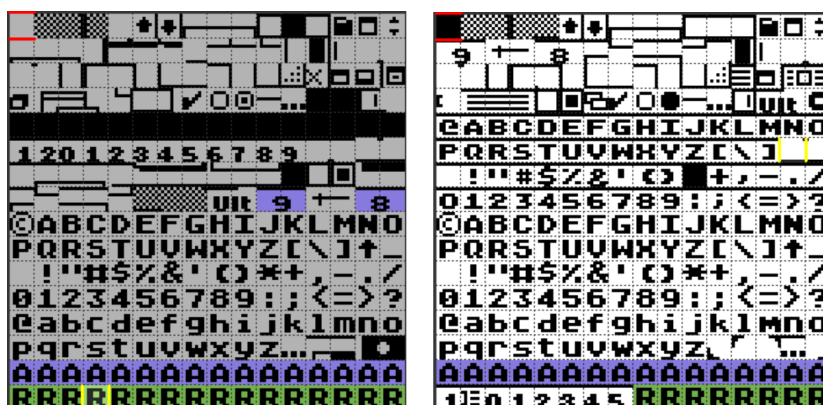
Dialog	API Call	Purpose
Message box	GUI_ShowMessage	Show single- or multi-line message text with an OK button
Error message box	GUI_ShowErrorMsg	Same as message box, but with title "Error"
Yes-No dialog	GUI_ShowYesNoDlg	Show single- or multi-line message text with Yes and No buttons. Title of dialog has to be provided. Has a return value

Custom Dialogs

An app may define its own dialogs. To create a dialog, first create a window by calling **GUI_CreateWindow** or **GUI_CreateWindowEx** as documented above, optionally make any required adjustments to the controls, and then call **GUI_ShowWndAsDlg**. This displays the window on screen and causes GUI64 to treat it as a dialog. To close the current dialog, call **GUI_KillDialog**. If a dialog, thus created, is closed via a click in its title bar, GUI64 cares for the correct dialog kill.

3 Character set

As most C64 software using character graphics mode, GUI64 uses a custom character set. The character sets of WIN and MAC design look as follows:



The bin files containing the character definitions can be found in the *Code* folder in the GUI64 Github repository <https://github.com/WebFritzi/GUI64>.

The 16 blue A's (indices 224 - 239) are at the same place in both designs. This area is reserved for custom characters defined by an app. An app registers new characters by calling **GUI_RegisterChars**. The address of the start of the blue A's area in the character set in memory is publicly available as the constant **GUI_CHARSET** (see **gui64.inc.asm**). For writing directly into the charset area, the app should call **GUI_MapOutIO** first. When it has finished writing, it should call **GUI_MapInIO**. However, this should never be done in the IRQ handler.

4 Two Designs

As there exist two designs of GUI64 (WIN design and MAC design), the programmer should always check whether the controls are placed as intended in both designs. For this, they can use **GUI_GetDesign**.

5 Sprites and SID

In both designs, the mouse pointer always occupies sprites 0 and 1. While there are no further sprites used in the MAC design, the WIN design uses sprites 2, 3, 4, and 5 for the Commodore button and the highlighting in the CBM 'start' menu. Therefore, if an app wants to make extensive use of sprites, it should be written exclusively for the MAC design and shut down immediately if the user is trying to load it in the WIN design:

```

*=$b000
        jsr GUI_GetDesign
        bne +
        ldx #<Str_Mess
        ldy #>Str_Mess
        jmp GUI_ShowMessage
+
        ...

Str_Mess    !pet "This app only works in the\MAC design of GUI64.",0

```

If an app is supposed to work with both designs, it should only use sprites 6 and 7.

As the VIC bank is in **\$c000 - \$ffff**, sprite definitions can be placed in the app area from **\$c000** onwards. For saving app RAM space, an app may alternatively copy sprite definitions to the area **\$de00 - \$dfff**.

The event code **EC_SHUTDOWN**

When an app shuts down, the event code **EC_SHUTDOWN** is sent to the app's main window proc. If an app uses sprites, it should react to this event code by turning off the respective bits in **\$D015**. Also, if an app plays music or sound effects via the **SID** chip, it should react to this event code by setting the SID volume register **\$D418** to zero.

6 Frame and Timer API

6.1 Frame API

The frame API provides applications with a callback that is executed once per video frame. It is intended for applications that require frame-synchronous processing, such as games, sprite animation or direct VIC-II effects.

The frame handler is called *directly from the GUI64 raster interrupt*. Consequently, the handler executes in interrupt context and must obey the corresponding restrictions.

The frame API consists of three routines:

- **GUI_SetFrameHandler** registers the application frame handler.
- **GUI_StartFrameHandling** enables frame callbacks.
- **GUI_StopFrameHandling** disables frame callbacks.

A typical setup is:

```

ldx #<FrameHandler
ldy #>FrameHandler
jsr GUI_SetFrameHandler
jsr GUI_StartFrameHandling
...
FrameHandler
; Short, frame-synchronous processing only.
rts

```

The frame handler must return with **rts**.

Warning: The frame handler runs inside the raster IRQ. It must remain short and must not call general GUI64 window, control, dialog or repaint routines. Such routines are not guaranteed to be reentrant and may use GUI64 working memory that was already in use when the interrupt occurred.

GUI64 uses shared internal state including the current window and control structures as well as zero-page working pointers. Calling a non-IRQ-safe API routine from the frame handler could modify this state while an interrupted GUI64 routine still depends on it.

Only API functions explicitly documented as IRQ-safe may therefore be called from a frame handler. In particular, routines that select or update controls, create or remove windows, repaint the GUI, display dialogs or manipulate GUI64 scratch pointers must not be called from frame context. As of *GUI64 API 1.0*, the following API routines are IRQ-safe:

- `GUI_GetDesign`
- `GUI_GetCurMenuID`
- `GUI_GetCSTMWindowColor`
- `GUI_GetCSTMDesktopColor`
- `GUI_GetScreenMem`
- `GUI_StopFrameHandling`

The application should also avoid lengthy calculations, blocking loops and other operations that could delay subsequent raster interrupts. A frame handler that runs for too long may disturb raster-sensitive GUI64 operations, including display and taskbar processing.

For work that does not require exact frame synchronization, the timer API or normal application event processing should be preferred.

If an app uses frame handling, it should react to the `EC_SHUTDOWN` event code in the main window proc by calling `GUI_StopFrameHandling`.

6.2 Timer API

The GUI64 timer provides applications with periodic callbacks at one-tenth-second intervals. The timer is based on the CIA time-of-day (TOD) clock.

Unlike the frame callback, the application timer handler is *not* executed directly from the raster interrupt. When a timer tick occurs, GUI64 records a timer event. The event is subsequently dispatched by the GUI64 main loop, where the registered application timer handler is called in normal application context.

This distinction is important: because the timer handler does not execute in interrupt context, it may use the regular GUI64 API, including window, control and repaint functions.

The timer API consists of three routines:

- `GUI_InitTimer` registers the application timer handler.
- `GUI_StartTimer` enables periodic timer events.
- `GUI_StopTimer` disables timer events.

A typical timer setup is:

```
ldx #<TimerHandler
ldy #>TimerHandler
jsr GUI_InitTimer
jsr GUI_StartTimer
...
TimerHandler
; Periodic application processing.
; Regular GUI64 API calls are allowed here.
rts
```

The timer handler should return with `rts`.

If an app uses the timer, it should react to the `EC_SHUTDOWN` event code in the main window proc by calling `GUI_StopTimer`.

Note: Timer events are generated at one-tenth-second intervals, but the handler is executed by the GUI64 main loop. Its actual execution may therefore be delayed while GUI64 is processing another event or operation.

6.3 Timer and Frame Handler Comparison

Property	Timer handler	Frame handler
Execution context	GUI64 main loop	Raster IRQ

Callback rate	Every 1/10 second	Once per video frame
Raster synchronized	No	Yes
GUI64 API calls	Regular API calls are allowed	Only API-functions explicitly documented as IRQ-safe may be called
Typical use	Periodic application tasks, status updates and UI changes	Games, animation, and other frame-synchronous processing
Return instruction	rts	rts

Note: Use the timer API for ordinary periodic application work. Use the frame API only when processing must be synchronized with the video frame.

7 Public Jump Table

The public GUI64 API is exposed through a fixed jump table. Each entry consists of a three-byte **JMP** instruction that transfers control to the corresponding GUI64 routine. Applications should call these public entry points instead of internal GUI64 routines.

The table below lists all currently available public API entries, their fixed addresses, their functional group and a short description. More detailed information about parameters, return values and usage can be found in the following section.

Address	Entry point	Group	Purpose
\$5F00	GUI_CreateWindow	Window	Create a window from a static window definition
\$5F03	GUI_CreateWindowEx	Window	Create a window including the controls that follow the window definition
\$5F06	GUI_KillCurWindow	Window	Remove the current window and its controls
\$5F09	GUI_StdWndProc	Events	Run GUI64 standard window event processing
\$5F0C	GUI_IsInCurControl	Control	Test whether the mouse position is inside the current control
\$5F0F	GUI_Repaint	System	Repaint the GUI
\$5F12	GUI_GetDesign	System	Return the active GUI64 design variant (WIN/MAC)
\$5F15	GUI_UpdateControl	Control	Write the current static control structure back to the control heap
\$5F18	GUI_SelectCtrl_AddBits	Control	MAC helper for selected-control flags; just selects control in the WIN build
\$5F1B	GUI_SetCtrlStringList	Control	Assign a string-list pointer and number of strings to the current control
\$5F1E	GUI_SetCtrlString	Control	Assign a string pointer to the current control
\$5F21	GUI_SelectControl	Control	Select a control by its index
\$5F24	GUI_SelectControl0	Control	Select control with index 0
\$5F27	GUI_SelectControl1	Control	Select control with index 1
\$5F2A	GUI_SelectControl2	Control	Select control with index 2
\$5F2D	GUI_SelectControl3	Control	Select control with index 3
\$5F30	GUI_SelectControl4	Control	Select control with index 4
\$5F33	GUI_UpdateWindow	Window	Write the current static window structure back to the window heap
\$5F36	GUI_SetEditSLInfo	Control	Set single-line edit control information
\$5F39	GUI_RegisterChars	App	Register up to 16 application-defined characters
\$5F3C	GUI_SetCtrlActionsRoutine	App	Register the application control-event routine for newly defined controls
\$5F3F	GUI_SetPaintCtrlsRoutine	App	Register the application control-paint routine for newly defined controls
\$5F42	GUI_AddBufWidthToFD	Low-level	Move paint cursor in \$FD/\$FE down by 1
\$5F45	GUI_AddBufWidthTo02	Low-level	Move color cursor in \$02/\$03 down by 1
\$5F48	GUI_IsInCurMenu	Menu	Test whether the mouse position is inside the current menu

Address	Entry point	Group	Purpose
\$5F4B	GUI_GetCurMenuID	Menu	Return the current menu ID
\$5F4E	GUI_ShowMessage	Dialog	Show a message dialog box
\$5F51	GUI_GetMousePosInWnd	Mouse	Update MousePosInWndX/Y with coordinates relative to the current window's control area
\$5F54	GUI_GetCSTMWindowColor	System	Return the custom window color
\$5F57	GUI_GetCSTMDesktopColor	System	Return the custom desktop color
\$5F5A	GUI_StartTimer	Timer	Start the application timer; ticks are generated every 1/10 second
\$5F5D	GUI_StopTimer	Timer	Stop the application timer
\$5F60	GUI_InitTimer	Timer	Register the application timer handler address
\$5F63	GUI_RepaintCurWindow	Window	Paints the current window
\$5F66	GUI_FindWndByType	Window	Looks for a window of window type in Param0
\$5F69	GUI_SelectTopWindow	Window	Selects window with handle in Param0 and makes it the top window
\$5F6C	GUI_AddControl	Control	Adds a control to the current window
\$5F6F	GUI_SetFrameHandler	Frame	Register the frame handler address
\$5F72	GUI_StartFrameHandling	Frame	Start frame handling
\$5F75	GUI_StopFrameHandling	Frame	Stop frame handling
\$5F78	GUI_MapOutIO	System	Makes \$d000 - \$dfff RAM
\$5F7B	GUI_MapInIO	System	Makes \$d000 - \$dfff IO
\$5F7E	GUI_GetScreenMem	System	Retrieve the high byte of the screen memory address in A
\$5F81	GUI_PaintBox	Control	Paint a box inside a control
\$5F84	GUI_ShowYesNoDlg	Dialog	Show a message box with yes and no button
\$5F87	GUI_ShowErrorMsg	Dialog	Same as GUI_ShowMessage , but with title "Error"
\$5F8A	GUI_SetModalAddress	Dialog	Sets the modal address of Yes/No dialog
\$5F8D	GUI_GetDialogResult	Dialog	Retrieves user decision upon Yes/No dialog
\$5F90	GUI_ShowWndAsDlg	Dialog	Display created window as dialog on screen
\$5F93	GUI_KillDialog	Dialog	Kill current dialog

8 Documentation of API Calls

8.1 Window API

GUI_CreateWindow \$5F00

Creates a window without controls.

INPUT

Reg.	Value	Meaning
X	address low	Low byte of the static window-definition table
Y	address high	High byte of the static window-definition table

The window-definition table should contain the bytes: window type, bits, xPos, yPos, width, height, pointer to title string, and pointer to window proc.

```

ldx #<Wnd_Data
ldy #>Wnd_Data
jsr GUI_CreateWindow
...
Wnd_Data !byte WT_MyWnd, %00100001,11,2,11,15, <Str_Title, >Str_Title, <MyWndProc, >MyWndProc

```

GUI_CreateWindowEx \$5F03

Creates a window with controls.

INPUT

Reg.	Value	Meaning
X	address low	Low byte of the static window-definition table
Y	address high	High byte of the static window-definition table

The window-definition table should contain the bytes: window type, bits, xPos, yPos, width, height, pointer to title string, and pointer to window proc, followed by the static control data and a termination zero byte.

```
ldx #<Wnd_Data
ldy #>Wnd_Data
jsr GUI_CreateWindowEx
...
Wnd_Data !byte WT_MyWnd, %00100001,11,2,11,15, <Str_Title, >Str_Title, <MyWndProc, >MyWndProc
; Controls list
!byte CT_MENUBAR, <Menubar, >Menubar, 0, 0
!pet 0
!byte CT_LABEL, 1, 1, 11, 1
!pet "Hello world",0
!byte CT_BUTTON, 6, 0, 5, 3
!pet " OK ",0
; termination zero byte
!byte 0
```

GUI_KillCurWindow \$5F06

Removes the current window and its controls from window and control heap.

Warning: This call uses multiple GUI64 zero-page scratch locations. Do not keep an application pointer in temporary GUI64 zero-page storage across this call.

GUI_StdWndProc \$5F09

Runs the standard GUI64 window proc: updates the mouse position relative to the current window and then dispatches standard behavior for mouse, joystick and keyboard events.

This routine should be called in each window proc.

```
MyWndProc jsr GUI_StdWndProc

; application-specific processing follows
rts
```

GUI_UpdateWindow \$5F33

Copies the 16-byte static current-window structure in **\$10 - \$1f** to the window heap.

```
lda #20
sta WindowWidth
jsr GUI_UpdateWindow
```

GUI_RepaintCurWindow \$5F63

Repaints the current window on screen.

GUI_FindWndByType \$5F66

Looks for a window of window type in **Param0**.

OUTPUT

Result	Meaning
Carry set	Window found
Carry clear	Window not found

If found, the handle of the window is in **X**.

GUI_SelectTopWindow \$5F69

Selects window with handle in **Param0** and makes it the top window.

8.2 Control API

GUI_IsInCurControl \$5F0C

Tests whether MousePosInWndX/Y lies inside the current control.

OUTPUT

Result	Meaning
Carry set	Mouse is inside the current control
Carry clear	Mouse is outside the current control

GUI_UpdateControl \$5F15

Copies the 16-byte static current-control structure in **\$20 - \$2f** to the control heap.

```
lda #CL_WHITE
sta ControlColor
jsr GUI_UpdateControl
```

GUI_SelectCtrl_AddBits \$5F18

MAC: Selects control (see **GUI_SelectControl**) with index in **Y** and adds the control bits in **A**
 WIN: Selects control (see **GUI_SelectControl**) with index in **Y**

GUI_SetCtrlStringList \$5F1B

Assigns a string-list address and its item count to the current control, then updates the heap entry. Hence, no **GUI_UpdateControl** call necessary.

INPUT

Reg.	Value	Meaning
A	count	Number of strings in the list

X	address low	String-list address, low byte
Y	address high	String-list address, high byte

```
jsr GUI_SelectControl0
ldx #<Str_Menu
ldy #>Str_Menu
lda #2
jsr GUI_SetCtrlStringList
```

GUI_SetCtrlString \$5F1E

Assigns a string address to the current control and updates the heap entry. Hence, no **GUI_UpdateControl** call necessary.

INPUT

Reg.	Value	Meaning
X	address low	String address, low byte
Y	address high	String address, high byte

```
ldx #<Str_Label
ldy #>Str_Label
jsr GUI_SetCtrlString
```

GUI_SelectControl \$5F21

Selects a control by index and loads its heap entry into the static control structure in **\$20 - \$2f**.

INPUT

Reg.	Value
A	Control index

```
lda #5
jsr GUI_SelectControl
```

The convenience entries **GUI_SelectControl0** through **GUI_SelectControl4** select indices 0 through 4 without requiring the index in **A**.

GUI_AddControl \$5F6C

Adds a control to the current window and selects it.

INPUT

Reg.	Value	Meaning
X	address low	Control information data, low byte
Y	address high	Control information data, high byte

8.3 Application Extensions

GUI_RegisterChars \$5F39

Copies application-defined character data into the application-character area in the GUI64 character set.

INPUT

Reg.	Value	Meaning
A	1-16	Number of characters
X	address low	Character table address, low byte
Y	address high	Character table address, high byte

Values of 17 or greater are rejected by returning without copying.

```
ldx #<CharList
ldy #>CharList
lda #14
jsr GUI_RegisterChars
```

GUI_SetCtrlActionsRoutine \$5F3C

Registers the application routine that handles events for application-defined controls.

INPUT

Reg.	Value	Meaning
X	address low	Handler address, low byte
Y	address high	Handler address, high byte

Once the program pointer enters the provided actions routine, the **X** register is the control type.

```
ldx #<MyCtrlAction
ldy #>MyCtrlAction
jsr GUI_SetCtrlActionsRoutine
```

GUI_SetPaintCtrlsRoutine \$5F3F

Registers the application routine used to paint application-defined controls.

INPUT

Reg.	Value	Meaning
X	address low	Paint routine address, low byte
Y	address high	Paint routine address, high byte

Once the program pointer enters the provided paint routine, the **X** register is the control type. Moreover, **\$fd/\$fe** points to the position of the control in the paint buffer, and **\$02/\$03** points to the position in the color buffer.

```
ldx #<MyCtrlPaintRoutine
ldy #>MyCtrlPaintRoutine
jsr GUI_SetPaintCtrlsRoutine
```

8.4 Timer API

GUI_InitTimer \$5F60

Registers the application's timer handler.

INPUT

Reg.	Value	Meaning
X	address low	Timer-handler address, low byte
Y	address high	Timer-handler address, high byte

```
ldx #<MyTimerHandler
ldy #>MyTimerHandler
jsr GUI_InitTimer
```

GUI_StartTimer \$5F5A

Starts the application timer. GUI64 generates timer events at one-tenth-second granularity from the CIA TOD clock.

GUI_StopTimer \$5F5D

Stops the application timer.

8.5 Frame API

GUI_SetFrameHandler \$5F6F

Registers the application's frame handler.

INPUT

Reg.	Value	Meaning
X	address low	Frame-handler address, low byte
Y	address high	Frame-handler address, high byte

```
ldx #<MyFrameHandler
ldy #>MyFrameHandler
jsr GUI_SetFrameHandler
```

GUI_StartFrameHandling \$5F72

Starts frame handling.

GUI_StopFrameHandling \$5F75

Stops frame handling.

8.6 System API

GUI_Repaint \$5F0F

Repaints the entire GUI.

GUI_GetDesign \$5F12

Determines the currently used GUI64 design.

OUTPUT zero flag (Z=1 (**beq**): WIN, Z=0 (**bne**): MAC)

GUI_GetCSTMWindowColor \$5F54

OUTPUT

Reg.	Value
A	Standard window color

GUI_GetCSTMDesktopColor \$5F57

OUTPUT

Reg.	Value
A	Standard desktop color

GUI_MapOutIO \$5F78

Declares **\$D000 - \$DFFF** as RAM. The app then has access to the character set in **\$D000 - \$D7FF** and sprites in **\$DE00 - \$DFFF**.

GUI_MapInIO \$5F7B

Should be called after a call to **GUI_MapOutIO**. Declares **\$D000 - \$DFFF** as IO area.

GUI_GetScreenMem \$5F7E

Returns the high byte of the screen memory address in **A**. The low byte is always **\$00**.

8.7 Menu and Mouse API

GUI_GetCurMenuID \$5F4B

OUTPUT

Reg.	Value
A	menu ID

```
jsr GUI_GetCurMenuID
cmp #ID_MENU_GAME
beq GameMenuClicked
```

GUI_GetMousePosInWnd \$5F51

Updates **MousePosInWndX** and **MousePosInWndY** with coordinates relative to the area of the current window in which controls are placed.

8.8 Dialog API

GUI_ShowMessage \$5F4E

Shows a message dialog box with text in **Addr(X/Y)**, an OK button below it, and “Info” in the titlebar. The text is placed in a multi-line label (**CT_LABEL_ML**). Line feeds are indicated by the backslash symbol (**\$1c** in memory).

```
ldx #<MyMessage
ldy #>MyMessage
jsr GUI_ShowMessage
...
MyMessage !pet "I am an info box.\Just click OK.",0
```

GUI_ShowErrorMsg \$5F87

Same as **GUI_ShowMessage**, but with the title “Error”.

GUI_ShowYesNoDlg \$5F84

Shows a message box dialog with two buttons (Yes and No) and text in **Addr(X/Y)**. The title has to be specified as a string pointer in **\$fd/\$fe**.

Before the call, the app has to specify an address, the program counter should jump to after the user’s decision. For this, use **GUI_SetModalAddress** (see below). In the respective routine, the app retrieves the user’s decision via **GUI_GetDialogResult** (see below).

```
ldx #<ModalAddr
ldy #>ModalAddr
jsr GUI_SetModalAddress
ldx #<MyMessage
ldy #>MyMessage
jmp GUI_ShowYesNoDlg
...
ModalAddr jsr GUI_GetDialogResult
bne yes
; user chose no
...
yes ; user chose yes
...
...
MyQuestion !pet "Do you want this?",0
```

GUI_SetModalAddress \$5F8A

Specifies the (modal) address, the program counter should jump to after the user’s decision in Yes/No dialog.

GUI_GetDialogResult \$5F8D

Retrieves the user's decision upon question in Yes/No dialog.

OUTPUT zero flag (Z=1 (**beq**): Yes, Z=0 (**bne**): No)

GUI_ShowWndAsDialog \$5F90

Sets **ProgramMode** to **PM_DIALOG** and shows the dialog (current window) on screen.

GUI_KillDialog \$5F93

Kills the dialog (current window) and sets **ProgramMode** back to **PM_NORMAL**.

8.9 Low-level Buffer Helpers

The following public entries expose GUI64 pointer arithmetic. They are useful for custom controls, but unlike higher-level calls they operate on GUI64's scratch pointers directly.

Entry point	Operation
GUI_AddBufWidthToFD	Move position \$fd/\$fe down by 1 in paint buffer (increment y coord)
GUI_AddBufWidthTo02	Move position \$02/\$03 down by 1 in color buffer (increment y coord)

Warning: These pointers are GUI64 working storage. Their contents are not preserved by unrelated GUI calls.

9 Hello World Application

The following application shows a window with a label displaying the text *Hello World*.

```
!source "gui64.inc.asm"

WT_MYWINDOW = 50

*=$b000
    ldx #<MyWindow
    ldy #>MyWindow
    jmp GUI_CreateWindowEx

MyWndProc    jmp GUI_StdWndProc

MyWindow
    !byte WT_MYWINDOW,BIT_WND_CANMINIMIZE,5,5,15,10
    !word Str_Title,MyWndProc
    !byte CT_LABEL,1,1,11,1
    !pet "Hello World",0
    ; Termination zero byte
    !byte 0
Str_Title    !pet "Hi",0
```